SpatialPolygonsDataFrame, Neighborhood Structure, Spatial Correlation and Regression

Poverty Data Example for Wisconsin and Illinois

Mahedi Hasan and Jun Zhu

SpatialPolygonsDataFrame

In this demonstration, we will work with a spatial polygon data object to create a neighborhood structure object.

In older R workflows, this type of object was often called a `SpatialPolygonsDataFrame` and was commonly handled using the `rgdal` and `sp` packages. In current R workflows, the modern package `sf` is preferred. Therefore, we will create the county-level spatial object using `sf`, then use it to construct a neighborhood structure object.

If the original shapefile is not available, we ca create it by joining the poverty dataset to county boundary polygons for Wisconsin and Illinois.

We will use the following packages: `sf`, `dplyr`, `readr`, `stringr`, `tigris`, and `spdep`.

Creating the Shape File

We first enable caching for the `tigris` package so that downloaded boundary files are stored locally and can be reused.

Next, we read the poverty dataset from the file `uspoverty2000.csv`. We then ensure that the county FIPS code is stored as a 5-digit character string by padding with leading zeros if needed.

```
options(tigris_use_cache = TRUE)

povdf <- read_csv("uspoverty2000.csv", show_col_types = FALSE)

povdf <- povdf %>%
  mutate(FIPS_n = str_pad(as.character(FIPS_N), width = 5, side = "left", pad = "0"))

str(povdf)
```

```
## tibble [3,070 x 13] (S3: tbl_df/tbl/data.frame)
##  $ FIPS_N  : num [1:3070] 1001 1003 1005 1007 1009 ...
##  $ ag      : num [1:3070] 2.13 1.57 3.97 3.3 2.18 ...
##  $ black   : num [1:3070] 17.11 10.29 46.32 22.2 1.19 ...
##  $ feemp   : num [1:3070] 53.2 50.5 43.5 44.3 48.1 ...
##  $ foreign : num [1:3070] 1.17 2.11 1.5 0.43 3.1 ...
##  $ hisp    : num [1:3070] 1.4 1.76 1.65 1.01 5.33 ...
##  $ hsch    : num [1:3070] 33.8 29.6 32.4 35.7 36 ...
##  $ manu    : num [1:3070] 16.5 12.5 31.4 23.8 19.5 ...
##  $ povty   : num [1:3070] 9.06 9.13 21.91 17.74 9.89 ...
##  $ retail  : num [1:3070] 12.85 14.2 10.51 9.93 11.61 ...
```

```
##  $ STATEFP : num [1:3070] 1 1 1 1 1 1 1 1 1 1 ...
##  $ COUNTYFP: num [1:3070] 1 3 5 7 9 11 13 15 17 19 ...
##  $ FIPS_n  : chr [1:3070] "01001" "01003" "01005" "01007" ...
```

```
head(povdf)
```

```
## # A tibble: 6 x 13
##   FIPS_N    ag black feemp foreign  hisp  hsch  manu povty retail STATEFP
##    <dbl> <dbl> <dbl> <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>   <dbl>
## 1   1001  2.13 17.1   53.2   1.17   1.40  33.8  16.5  9.06  12.9        1
## 2   1003  1.57 10.3   50.5   2.11   1.76  29.6  12.5  9.13  14.2        1
## 3   1005  3.97 46.3   43.5   1.5    1.65  32.4  31.4 21.9   10.5        1
## 4   1007  3.30 22.2   44.3   0.430  1.01  35.7  23.8 17.7    9.93       1
## 5   1009  2.18  1.19  48.1   3.10   5.33  36.0  19.5  9.89  11.6        1
## 6   1011 10.5  73.1   36.5   3.06   2.75  35.2  23.2 28.6    7.53       1
## # i 2 more variables: COUNTYFP <dbl>, FIPS_n <chr>
```

We next obtain county boundaries for Wisconsin and Illinois. The function `counties()` from the `tigris` package returns county polygons as an `sf` object in modern R workflows.

```
# Get county boundaries for Wisconsin and Illinois
# tigris::counties() returns sf by default in modern versions
wi_counties <- counties(state = "WI", cb = FALSE, year = 2024)
il_counties <- counties(state = "IL", cb = FALSE, year = 2024)

counties_wi_il <- bind_rows(wi_counties, il_counties)

str(counties_wi_il)
```

```
## Classes 'sf' and 'data.frame':   174 obs. of  19 variables:
##  $ STATEFP : chr  "55" "55" "55" "55" ...
##  $ COUNTYFP: chr  "111" "093" "063" "033" ...
##  $ COUNTYNS: chr  "01581115" "01581107" "01581091" "01581076" ...
##  $ GEOID   : chr  "55111" "55093" "55063" "55033" ...
##  $ GEOIDFQ : chr  "0500000US55111" "0500000US55093" "0500000US55063" "0500000US55033" ...
##  $ NAME    : chr  "Sauk" "Pierce" "La Crosse" "Dunn" ...
##  $ NAMELSAD: chr  "Sauk County" "Pierce County" "La Crosse County" "Dunn County" ...
##  $ LSAD    : chr  "06" "06" "06" "06" ...
##  $ CLASSFP : chr  "H1" "H1" "H1" "H1" ...
##  $ MTFCC   : chr  "G4020" "G4020" "G4020" "G4020" ...
##  $ CSAFP   : chr  "357" "378" "317" "232" ...
##  $ CBSAFP  : chr  "12660" "33460" "29100" "32860" ...
##  $ METDIVFP: chr  NA NA NA NA ...
##  $ FUNCSTAT: chr  "A" "A" "A" "A" ...
##  $ ALAND   : num  2.15e+09 1.49e+09 1.17e+09 2.20e+09 2.56e+09 ...
##  $ AWATER  : num  45578927 48452323 73160503 35572579 32538319 ...
##  $ INTPTLAT: chr  "+43.4280010" "+44.7253365" "+43.9082222" "+44.9477513" ...
##  $ INTPTLON: chr  "-089.9433184" "-092.4262793" "-091.1117584" "-091.8976451" ...
##  $ geometry:sfc_MULTIPOLYGON of length 174; first list element: List of 1
##   ..$ :List of 1
##   .. ..$ : num [1:2137, 1:2] -89.7 -89.7 -89.7 -89.7 -89.7 ...
##   ..- attr(*, "class")= chr [1:3] "XY" "MULTIPOLYGON" "sfg"
##  - attr(*, "sf_column")= chr "geometry"
##  - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",..: NA NA NA NA NA NA NA NA NA NA ...
##   ..- attr(*, "names")= chr [1:18] "STATEFP" "COUNTYFP" "COUNTYNS" "GEOID" ...
```

We then create a common county identifier called `FIPS_n` from the county `GEOID` so that the poverty dataset and the county boundary dataset can be merged.

```
# Keep and align county identifier
counties_wi_il <- counties_wi_il %>%
  mutate(FIPS_n = GEOID)

head(counties_wi_il)
```

```
## Simple feature collection with 6 features and 19 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -92.80842 ymin: 43.14529 xmax: -88.88518 ymax: 45.20957
## Geodetic CRS:  NAD83
##   STATEFP COUNTYFP COUNTYNS GEOID        GEOIDFQ       NAME          NAMELSAD
## 1      55      111 01581115 55111 0500000US55111       Sauk       Sauk County
## 2      55      093 01581107 55093 0500000US55093     Pierce     Pierce County
## 3      55      063 01581091 55063 0500000US55063  La Crosse  La Crosse County
## 4      55      033 01581076 55033 0500000US55033       Dunn       Dunn County
## 5      55      053 01581086 55053 0500000US55053    Jackson    Jackson County
## 6      55      047 01581083 55047 0500000US55047 Green Lake Green Lake County
##   LSAD CLASSFP MTFCC CSAFP CBSAFP METDIVFP FUNCSTAT      ALAND   AWATER
## 1   06      H1 G4020   357  12660     <NA>        A 2153800051 45578927
## 2   06      H1 G4020   378  33460     <NA>        A 1486700534 48452323
## 3   06      H1 G4020   317  29100     <NA>        A 1170164969 73160503
## 4   06      H1 G4020   232  32860     <NA>        A 2201902873 35572579
## 5   06      H1 G4020  <NA>   <NA>     <NA>        A 2558687748 32538319
## 6   06      H1 G4020  <NA>   <NA>     <NA>        A  905306376 79211051
##      INTPTLAT     INTPTLON                       geometry FIPS_n
## 1 +43.4280010 -089.9433184 MULTIPOLYGON (((-89.71636 4...  55111
## 2 +44.7253365 -092.4262793 MULTIPOLYGON (((-92.37492 4...  55093
## 3 +43.9082222 -091.1117584 MULTIPOLYGON (((-91.21186 4...  55063
## 4 +44.9477513 -091.8976451 MULTIPOLYGON (((-92.15705 4...  55033
## 5 +44.3245897 -090.7995104 MULTIPOLYGON (((-90.31752 4...  55053
## 6 +43.7802469 -088.9703796 MULTIPOLYGON (((-89.00685 4...  55047
```

Next, we join the poverty data to the county polygons using `FIPS_n`. This creates the spatial object that contains both the county-level geometry and the poverty variables.

```
# Join your poverty data to county polygons
ilwipov_shape <- counties_wi_il %>%
  left_join(povdf, by = "FIPS_n")

class(ilwipov_shape)
```

```
## [1] "sf"         "data.frame"
```

```
names(ilwipov_shape)
```

```
##  [1] "STATEFP.x"  "COUNTYFP.x" "COUNTYNS"   "GEOID"      "GEOIDFQ"
##  [6] "NAME"       "NAMELSAD"   "LSAD"       "CLASSFP"    "MTFCC"
## [11] "CSAFP"      "CBSAFP"     "METDIVFP"   "FUNCSTAT"   "ALAND"
## [16] "AWATER"     "INTPTLAT"   "INTPTLON"   "FIPS_n"     "FIPS_N"
## [21] "ag"         "black"      "feemp"      "foreign"    "hisp"
## [26] "hsch"       "manu"       "povty"      "retail"     "STATEFP.y"
## [31] "COUNTYFP.y" "geometry"
```

As a quick diagnostic, we can check how many counties did not match the poverty variable `povty`.

```
# Optional check: see whether any counties did not match
sum(is.na(ilwipov_shape$povty))
```

```
## [1] 0
```

We now save the spatial object as a shapefile so that it can be reused later without recreating it from scratch.

```
st_write(ilwipov_shape, "ilwipov_shp.shp", delete_layer = TRUE, quiet = TRUE)
```

Exploring the Spatial Object

We next check the class of the new object.

```
# You can now use it directly
class(ilwipov_shape)
```

```
## [1] "sf"         "data.frame"
```

We can see that `ilwipov_shape` is an `sf` object, which is the modern equivalent of the older polygon-based spatial data frame used in previous spatial workflows.

We use the `names()` function to see what variables are contained in the spatial object.

```
names(ilwipov_shape)
```

```
##  [1] "STATEFP.x"  "COUNTYFP.x" "COUNTYNS"   "GEOID"      "GEOIDFQ"
##  [6] "NAME"       "NAMELSAD"   "LSAD"       "CLASSFP"    "MTFCC"
## [11] "CSAFP"      "CBSAFP"     "METDIVFP"   "FUNCSTAT"   "ALAND"
## [16] "AWATER"     "INTPTLAT"   "INTPTLON"   "FIPS_n"     "FIPS_N"
## [21] "ag"         "black"      "feemp"      "foreign"    "hisp"
## [26] "hsch"       "manu"       "povty"      "retail"     "STATEFP.y"
## [31] "COUNTYFP.y" "geometry"
```

In the output above, we see that there are additional variables from the county boundary file, in addition to the poverty and socio-economic factors of interest.

We can inspect the first few observations using the `head()` function.

```
head(ilwipov_shape)
```

```
## Simple feature collection with 6 features and 31 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -92.80842 ymin: 43.14529 xmax: -88.88518 ymax: 45.20957
## Geodetic CRS:  NAD83
##   STATEFP.x COUNTYFP.x COUNTYNS GEOID        GEOIDFQ       NAME
## 1        55        111 01581115 55111 0500000US55111       Sauk
## 2        55        093 01581107 55093 0500000US55093     Pierce
## 3        55        063 01581091 55063 0500000US55063  La Crosse
## 4        55        033 01581076 55033 0500000US55033       Dunn
## 5        55        053 01581086 55053 0500000US55053    Jackson
## 6        55        047 01581083 55047 0500000US55047 Green Lake
##           NAMELSAD LSAD CLASSFP MTFCC CSAFP CBSAFP METDIVFP FUNCSTAT
## 1       Sauk County   06      H1 G4020   357  12660     <NA>        A
## 2     Pierce County   06      H1 G4020   378  33460     <NA>        A
## 3  La Crosse County   06      H1 G4020   317  29100     <NA>        A
## 4       Dunn County   06      H1 G4020   232  32860     <NA>        A
## 5    Jackson County   06      H1 G4020  <NA>   <NA>     <NA>        A
```

```
## 6 Green Lake County   06      H1 G4020  <NA>   <NA>     <NA>        A
##        ALAND   AWATER    INTPTLAT     INTPTLON FIPS_n FIPS_N   ag black feemp
## 1 2153800051 45578927 +43.4280010 -089.9433184  55111  55111 5.27  0.26 63.44
## 2 1486700534 48452323 +44.7253365 -092.4262793  55093  55093 4.77  0.25 70.33
## 3 1170164969 73160503 +43.9082222 -091.1117584  55063  55063 1.26  0.95 63.36
## 4 2201902873 35572579 +44.9477513 -091.8976451  55033  55033 7.11  0.34 62.84
## 5 2558687748 32538319 +44.3245897 -090.7995104  55053  55053 9.58  2.27 60.21
## 6  905306376 79211051 +43.7802469 -088.9703796  55047  55047 5.80  0.15 58.27
##   foreign hisp  hsch  manu povty retail STATEFP.y COUNTYFP.y
## 1    1.93 1.70 37.78 19.08  6.00  13.20        55        111
## 2    0.98 0.82 34.96 19.57  8.55  11.69        55         93
## 3    2.53 0.92 31.91 16.13 11.63  14.39        55         63
## 4    2.09 0.84 36.89 17.02 14.02  13.25        55         33
## 5    1.93 1.87 41.73 16.19  8.85  10.83        55         53
## 6    1.82 2.06 41.93 25.58  5.41  11.29        55         47
##                         geometry
## 1 MULTIPOLYGON (((-89.71636 4...
## 2 MULTIPOLYGON (((-92.37492 4...
## 3 MULTIPOLYGON (((-91.21186 4...
## 4 MULTIPOLYGON (((-92.15705 4...
## 5 MULTIPOLYGON (((-90.31752 4...
## 6 MULTIPOLYGON (((-89.00685 4...
```

Creating the Neighborhood Structure Object

In order to construct a neighborhood structure object from the county polygons, we use the `poly2nb()` function from the `spdep` package.

This function creates a neighborhood list from polygon boundaries. We specify `row.names = ilwipov_shape$FIPS_n` so that each slot in the neighborhood structure object is matched to the correct county.

```
# Neighborhood structure
ilwipov_nb <- poly2nb(ilwipov_shape, row.names = ilwipov_shape$FIPS_n)
```

To explore the neighborhood structure object `ilwipov_nb`, we check its class and structure.

```
class(ilwipov_nb)
```

```
## [1] "nb"
```

```
str(ilwipov_nb)
```

```
## List of 174
##  $ : int [1:6] 8 46 54 59 63 66
##  $ : int [1:3] 4 58 65
##  $ : int [1:4] 5 8 32 42
##  $ : int [1:7] 2 28 33 40 50 58 65
##  $ : int [1:7] 3 28 32 42 55 64 66
##  $ : int [1:6] 13 23 45 52 56 63
##  $ : int [1:7] 9 26 43 48 68 145 170
##  $ : int [1:6] 1 3 27 42 54 66
##  $ : int [1:4] 7 43 159 170
##  $ : int [1:4] 18 20 43 68
##  $ : int [1:4] 21 31 41 61
##  $ : int [1:5] 17 25 30 57 61
##  $ : int [1:6] 6 16 22 36 52 56
```

```
##  $ : int [1:6] 21 31 40 49 51 67
##  $ : int [1:5] 34 59 62 114 118
##  $ : int [1:6] 13 22 35 38 44 56
##  $ : int [1:4] 12 30 41 61
##  $ : int [1:4] 10 19 20 68
##  $ : int [1:5] 18 20 23 47 71
##  $ : int [1:6] 10 18 19 23 45 68
##  $ : int [1:4] 11 14 31 51
##  $ : int [1:6] 13 16 36 38 44 55
##  $ : int [1:6] 6 19 20 45 47 56
##  $ : int [1:3] 53 71 72
##  $ : int [1:7] 12 29 30 38 44 57 70
##  $ : int [1:6] 7 34 46 48 137 145
##  $ : int [1:3] 8 54 62
##  $ : int [1:7] 4 5 32 50 58 64 69
##  $ : int [1:7] 25 30 44 53 60 70 72
##  $ : int [1:6] 12 17 25 29 39 60
##  $ : int [1:7] 11 14 21 40 51 61 67
##  $ : int [1:4] 3 5 28 69
##  $ : int [1:4] 4 40 49 65
##  $ : int [1:6] 15 26 46 59 114 137
##  $ : int [1:5] 16 44 47 53 56
##  $ : int [1:6] 13 22 52 55 63 66
##  $ : int [1:6] 38 50 57 61 64 67
##  $ : int [1:8] 16 22 25 37 44 55 57 64
##  $ : int [1:2] 30 60
##  $ : int [1:8] 4 14 31 33 49 50 65 67
##  $ : int [1:3] 11 17 61
##  $ : int [1:4] 3 5 8 66
##  $ : int [1:4] 7 9 10 68
##  $ : int [1:8] 16 22 25 29 35 38 53 70
##  $ : int [1:7] 6 20 23 46 48 63 68
##  $ : int [1:7] 1 26 34 45 48 59 63
##  $ : int [1:6] 19 23 35 53 56 71
##  $ : int [1:5] 7 26 45 46 68
##  $ : int [1:4] 14 33 40 51
##  $ : int [1:6] 4 28 37 40 64 67
##  $ : int [1:4] 14 21 31 49
##  $ : int [1:4] 6 13 36 63
##  $ : int [1:7] 24 29 35 44 47 71 72
##  $ : int [1:5] 1 8 27 59 62
##  $ : int [1:6] 5 22 36 38 64 66
##  $ : int [1:6] 6 13 16 23 35 47
##  $ : int [1:5] 12 25 37 38 61
##  $ : int [1:4] 2 4 28 69
##  $ : int [1:6] 1 15 34 46 54 62
##  $ : int [1:4] 29 30 39 72
##  $ : int [1:8] 11 12 17 31 37 41 57 67
##  $ : int [1:5] 15 27 54 59 118
##  $ : int [1:7] 1 6 36 45 46 52 66
##  $ : int [1:6] 5 28 37 38 50 55
##  $ : int [1:4] 2 4 33 40
##  $ : int [1:7] 1 5 8 36 42 55 63
##  $ : int [1:6] 14 31 37 40 50 61
```

```
##  $ : int [1:7] 7 10 18 20 43 45 48
##  $ : int [1:3] 28 32 58
##  $ : int [1:3] 25 29 44
##  $ : int [1:4] 19 24 47 53
##  $ : int [1:4] 24 29 53 60
##  $ : int [1:4] 100 113 126 173
##  $ : int [1:6] 85 107 122 136 148 150
##  $ : int [1:4] 107 110 164 172
##  $ : int [1:8] 80 102 111 121 132 142 162 166
##  $ : int [1:4] 89 117 136 154
##  $ : int [1:5] 100 113 131 140 146
##  $ : int [1:4] 129 149 154 160
##  $ : int [1:6] 76 90 103 111 139 166
##  $ : int [1:4] 124 127 134 158
##  $ : int [1:7] 97 99 112 128 143 147 161
##  $ : int [1:6] 86 92 103 135 139 167
##  $ : int [1:5] 93 128 151 169 173
##  $ : int [1:5] 74 86 135 136 148
##  $ : int [1:5] 83 85 92 135 148
##  $ : int [1:4] 92 107 148 164
##  $ : int [1:7] 113 120 126 142 151 165 169
##  $ : int [1:7] 77 135 136 141 154 160 168
##  $ : int [1:7] 80 111 135 139 141 162 168
##  $ : int [1:6] 109 119 133 158 163 174
##  $ : int [1:5] 83 86 87 148 167
##  $ : int [1:3] 84 128 161
##  $ : int [1:6] 98 123 125 143 152 157
##  $ : int [1:5] 123 152 157 159 170
##  $ : int [1:4] 109 110 124 158
##  $ : int [1:5] 82 98 138 143 161
##  $ : int [1:8] 94 97 123 137 138 143 145 170
##  $ : int [1:5] 82 128 147 165 169
##   [list output truncated]
##  - attr(*, "class")= chr "nb"
##  - attr(*, "region.id")= chr [1:174] "1" "2" "3" "4" ...
##  - attr(*, "call")= language poly2nb(pl = ilwipov_shape, row.names = ilwipov_shape$FIPS_n)
##  - attr(*, "type")= chr "queen"
##  - attr(*, "sym")= logi TRUE
```

The output above tells us that the neighborhood structure object is of class `nb`.

In the structure of `ilwipov_nb`, the neighbors list tells us which counties are neighbors of one another according to the polygon boundaries.

Next, using the R function `nb2listw()` from the `spdep` package, we create the list of spatial weights using the row-standardized coding given by `style = "W"`.

```
listw_povW <- nb2listw(ilwipov_nb, style = "W")
```

We take a closer look at some of the spatial weights by viewing the weights for the first several counties.

```
head(listw_povW$weights)
```

```
## [[1]]
## [1] 0.1666667 0.1666667 0.1666667 0.1666667 0.1666667 0.1666667
##
## [[2]]
```

```
## [1] 0.3333333 0.3333333 0.3333333
##
## [[3]]
## [1] 0.25 0.25 0.25 0.25
##
## [[4]]
## [1] 0.1428571 0.1428571 0.1428571 0.1428571 0.1428571 0.1428571 0.1428571
##
## [[5]]
## [1] 0.1428571 0.1428571 0.1428571 0.1428571 0.1428571 0.1428571 0.1428571
##
## [[6]]
## [1] 0.1666667 0.1666667 0.1666667 0.1666667 0.1666667 0.1666667
```

With `style = "W"` for row-standardized weights, each set of weights for a county sums to 1, as expected.

Coordinates and Visualization

We can extract centroid coordinates from the county polygons into an object called `coords`. Since we are working with an `sf` object, we first compute the centroids and then extract their coordinates.

```
# Centroid coordinates for plotting neighbors
coords <- st_coordinates(st_centroid(ilwipov_shape))
```

```
## Warning: st_centroid assumes attributes are constant over geometries
```

```
str(coords)
```

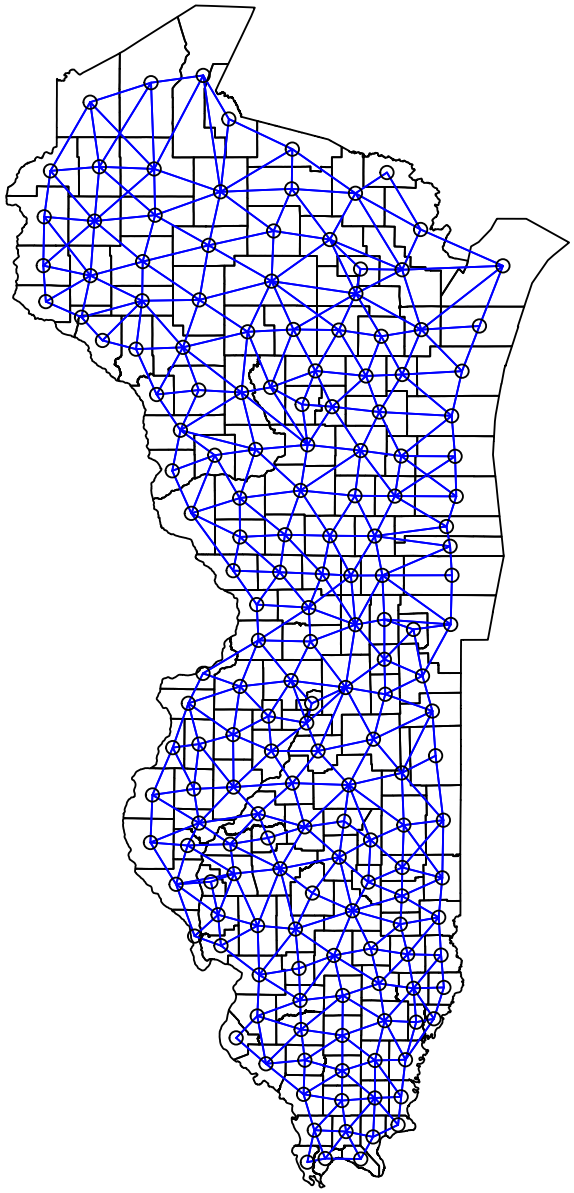
```
##  num [1:174, 1:2] -89.9 -92.4 -91.1 -91.9 -90.8 ...
##  - attr(*, "dimnames")=List of 2
##   ..$ : NULL
##   ..$ : chr [1:2] "X" "Y"
```

```
head(coords)
```

```
##              X        Y
## [1,] -89.94822 43.42652
## [2,] -92.42236 44.71941
## [3,] -91.11516 43.90647
## [4,] -91.89638 44.94627
## [5,] -90.80498 44.31919
## [6,] -89.04490 43.80025
```

We can visualize the neighbors by plotting the county polygons of Wisconsin and Illinois together with centroid connections showing their neighborhood relationships.

```
plot(st_geometry(ilwipov_shape),
     main = "WI and IL County Centroids and Second-Order Neighbors")
plot(listw_povW, coords, col = "blue", add = TRUE)
```


WI and IL County Centroids and Second−Order Neighbors

Additional Notes

The older function `readOGR()` from the `rgdal` package is no longer commonly available in recent R installations. In current spatial workflows, the `sf` package is the preferred alternative, and `st_read()` is typically used for reading shapefiles.

Spatial Correlation

In this demonstration, we will use Moran's I to explore spatial correlation in the Wisconsin/Illinois 2000 poverty dataset.

We first test the spatial correlation for `povty` and two of the explanatory variables, `feemp` and `black`.

We use `moran.test()` from the `spdep` package. The first argument is the variable we are interested in testing for spatial correlation, such as `povty`. The second argument is the list of spatial weights we created using row-standardized weights, `listw_povW`. Lastly, we specify the argument `alternative = "two.sided"` for a two-sided test. We perform this test under randomization, the default. We set the seed first.

Moran's I for `povty`

```
# Test spatial correlation in povty (response)
set.seed(1)
moran.test(ilwipov_shape$povty, listw_povW, alternative = "two.sided")
```

```
##
##  Moran I test under randomisation
##
## data:  ilwipov_shape$povty
## weights: listw_povW
##
## Moran I statistic standard deviate = 6.6835, p-value = 2.333e-11
## alternative hypothesis: two.sided
## sample estimates:
## Moran I statistic       Expectation          Variance
##       0.303599981      -0.005780347       0.002142768
```

For `povty`, we find that the Moran's I statistic is 0.304551888 and the variance is 0.002177963.

The standard deviate (z-value) is 6.6497. The p-value is 2.937 $\times$ 10^-11. We reject the null hypothesis of CSR at the 5% level. There is strong evidence for spatial dependence in `povty`.

We next explore the Moran's I statistic for `feemp`.

Moran's I for `feemp`

```
# Test spatial correlation in feemp (explanatory variable)
set.seed(2)
moran.test(ilwipov_shape$feemp, listw_povW, alternative = "two.sided")
```

```
##
##  Moran I test under randomisation
##
## data:  ilwipov_shape$feemp
## weights: listw_povW
##
## Moran I statistic standard deviate = 12.751, p-value < 2.2e-16
## alternative hypothesis: two.sided
## sample estimates:
## Moran I statistic       Expectation          Variance
##       0.591461322      -0.005780347       0.002193770
```

The explanatory variable `feemp` has a Moran's I statistic value of 0.587470384 and the p-value is less than 2.2 $\times$ 10^-16. We reject the null hypothesis of CSR at the 5% level. There is very strong evidence for spatial

dependence in `feemp`.

Lastly, we explore the Moran's I statistic for `black`.

Moran's I for `black`

```
# Test spatial correlation in black (explanatory variable)
set.seed(3)
moran.test(ilwipov_shape$black, listw_povW, alternative = "two.sided")
```

```
##
##  Moran I test under randomisation
##
## data:  ilwipov_shape$black
## weights: listw_povW
##
## Moran I statistic standard deviate = 5.2305, p-value = 1.69e-07
## alternative hypothesis: two.sided
## sample estimates:
## Moran I statistic       Expectation          Variance
##       0.230851139      -0.005780347       0.002046687
```

The explanatory variable `black` has a Moran's I statistic value of 0.230961391 and the p-value is 2.097 $\times$ 10^-7. We reject the null hypothesis of CSR at the 5% level. There is very strong evidence for spatial dependence in `black`.

Simple Linear Regression

Ordinary Least Squares

In this part, we fit a standard linear regression with multiple explanatory variables, but without any spatial consideration. We then use stepwise selection to find a parsimonious model.

First, we fit the full model with all 8 explanatory variables: `ag`, `manu`, `retail`, `foreign`, `feemp`, `hsch`, `black`, and `hisp`, with `povty` as the response. We use the `lm()` function to fit the model.

```
library(MASS)
```

```
##
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':
##
##     select
```

```
m1 <- lm(povty ~ ag + manu + retail + foreign + feemp + hsch + black + hisp,
         data = ilwipov_shape)
```

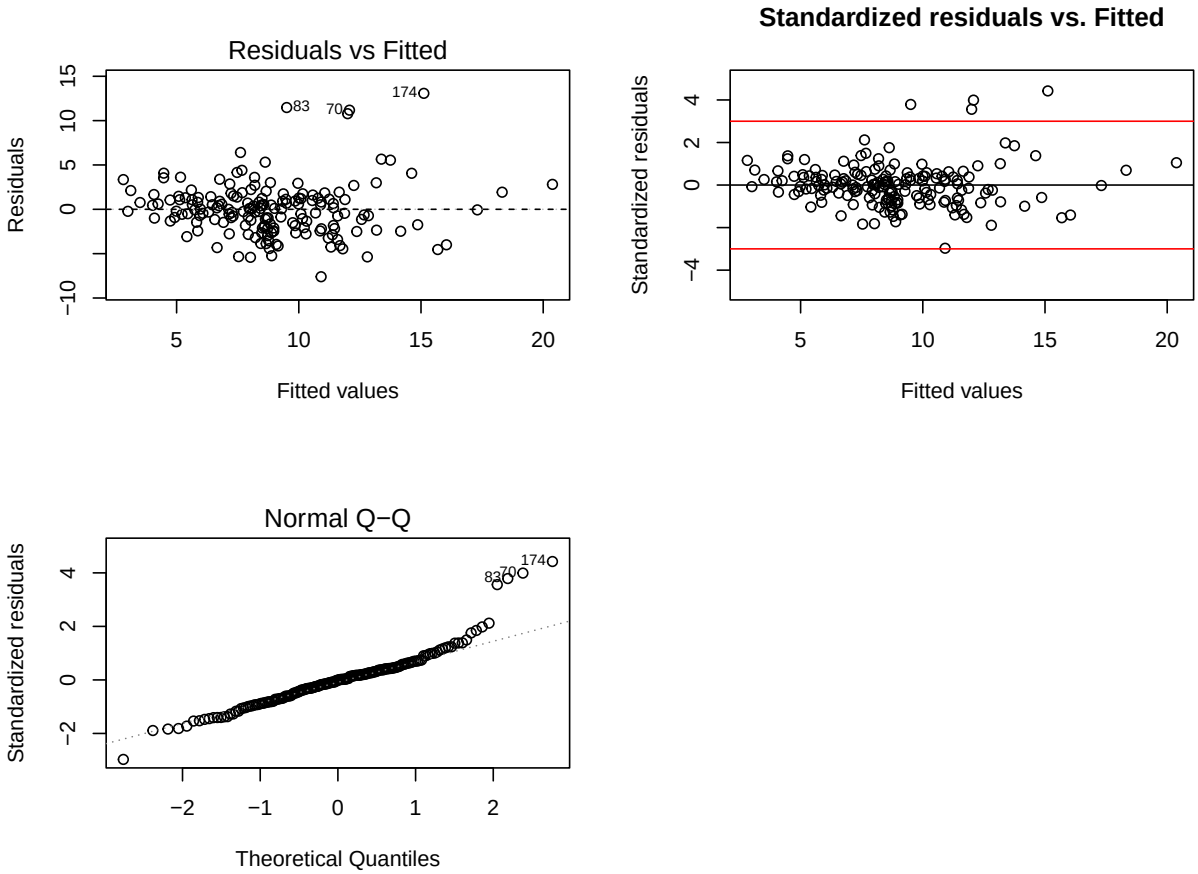
We perform model diagnostics using residual plots. We use the `stdres()` function from the `MASS` package to obtain standardized residuals.

```
par(mfrow = c(2, 2))

# Unstandardized residuals vs fitted
plot(m1, which = 1, add.smooth = FALSE)
abline(h = 0, lty = 2)

# Standardized residuals vs fitted
stdresids <- stdres(m1)
plot(x = fitted(m1), y = stdresids,
     xlab = "Fitted values",
     ylab = "Standardized residuals",
     main = "Standardized residuals vs. Fitted",
     ylim = c(-5, 5))
abline(h = c(-3, 0, 3), col = c("red", "black", "red"))

# QQ plot
plot(m1, which = 2, add.smooth = FALSE)
```



From the first plot of the (unstandardized) residuals versus fitted values, there is no obvious pattern that would indicate a violation of assumptions. However, R automatically identifies a few observations as potential outliers. In the standardized residuals versus fitted values plot, we also see some evidence of outlying observations. From the QQ-plot, the standardized residuals align reasonably well with the theoretical quantiles, except for possible outliers in the upper tail.

We use the `summary()` function to explore the model fit, `m1`.

```
summary(m1)
```

```
##
## Call:
## lm(formula = povty ~ ag + manu + retail + foreign + feemp + hsch +
##     black + hisp, data = ilwipov_shape)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -7.6031 -1.8360 -0.0301  1.2618 13.0744
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 34.11866    3.98805   8.555 7.75e-15 ***
## ag           0.12953    0.10085   1.284 0.200814
## manu        -0.15246    0.04383  -3.479 0.000644 ***
## retail      -0.09620    0.13363  -0.720 0.472612
```

```
## foreign      0.36065    0.21678   1.664 0.098084 .
## feemp       -0.31974    0.04984  -6.415 1.43e-09 ***
## hsch        -0.10548    0.07063  -1.493 0.137235
## black        0.18226    0.04865   3.746 0.000248 ***
## hisp        -0.46182    0.17379  -2.657 0.008652 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.062 on 165 degrees of freedom
## Multiple R-squared:  0.4926, Adjusted R-squared:  0.468
## F-statistic: 20.02 on 8 and 165 DF,  p-value: < 2.2e-16
```

From the summary output, we see that the fitted intercept is about 34.12. For `feemp`, the fitted regression coefficient is about -0.31974, with a very small p-value. This suggests strong evidence of a relationship between `feemp` and `povty`, after accounting for the other socio-economic factors.

We next test the model residuals for spatial correlation using Moran's I test and the list of spatial weights, `listw_povW`. We test by a normal approximation, specifying `randomisation = FALSE`.

```
moran.test(resid(m1), listw_povW, randomisation = FALSE, alternative = "two.sided")
```

```
##
##  Moran I test under normality
##
## data:  resid(m1)
## weights: listw_povW
##
## Moran I statistic standard deviate = 2.5547, p-value = 0.01063
## alternative hypothesis: two.sided
## sample estimates:
## Moran I statistic       Expectation          Variance
##       0.113854949      -0.005780347       0.002192984
```

From the output above, we find that the Moran's I test statistic is positive and the p-value is small. At the 5% level, we reject the null hypothesis. There is strong evidence that the model errors are not spatially independent.

Variable Selection

We now use stepwise selection to determine a subset of explanatory variables and select a reduced model.

Backward Elimination

We use stepwise regression to find a subset that minimizes AIC. We apply the `step()` function to the full model, `m1`, and specify that `direction = "backward"`.

```
m1_back <- step(m1, direction = "backward")
```

```
## Start:  AIC=398.23
## povty ~ ag + manu + retail + foreign + feemp + hsch + black +
##     hisp
##
##           Df Sum of Sq    RSS    AIC
## - retail   1      4.86 1552.2 396.78
## - ag       1     15.47 1562.8 397.96
## <none>                 1547.3 398.23
```

```
## - hsch     1     20.92 1568.2 398.57
## - foreign  1     25.95 1573.3 399.12
## - hisp     1     66.22 1613.5 403.52
## - manu     1    113.48 1660.8 408.55
## - black    1    131.59 1678.9 410.43
## - feemp    1    385.92 1933.2 434.97
##
## Step:  AIC=396.78
## povty ~ ag + manu + foreign + feemp + hsch + black + hisp
##
##           Df Sum of Sq    RSS    AIC
## - ag       1     17.35 1569.5 396.71
## <none>                 1552.2 396.78
## - hsch     1     20.46 1572.7 397.05
## - foreign  1     28.41 1580.6 397.93
## - hisp     1     67.62 1619.8 402.20
## - manu     1    108.68 1660.9 406.55
## - black    1    138.51 1690.7 409.65
## - feemp    1    418.49 1970.7 436.31
##
## Step:  AIC=396.71
## povty ~ manu + foreign + feemp + hsch + black + hisp
##
##           Df Sum of Sq    RSS    AIC
## - hsch     1      8.02 1577.6 395.60
## <none>                 1569.5 396.71
## - foreign  1     27.55 1597.1 397.74
## - hisp     1     70.64 1640.2 402.37
## - manu     1    131.71 1701.2 408.73
## - black    1    134.12 1703.7 408.98
## - feemp    1    407.27 1976.8 434.85
##
## Step:  AIC=395.6
## povty ~ manu + foreign + feemp + black + hisp
##
##           Df Sum of Sq    RSS    AIC
## <none>                 1577.6 395.60
## - foreign  1     43.18 1620.7 398.30
## - hisp     1     75.35 1652.9 401.72
## - black    1    177.90 1755.5 412.19
## - manu     1    188.44 1766.0 413.23
## - feemp    1    399.26 1976.8 432.85
```

The output from `step()` reports the variables selected at each stage of backward elimination, along with the AIC. The final selected model is shown at the end.

We can further explore the final selected model by applying `summary()` to `m1_back`.

```
summary(m1_back)
```

```
##
## Call:
## lm(formula = povty ~ manu + foreign + feemp + black + hisp, data = ilwipov_shape)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
```

```
## -7.3269 -1.8913 -0.1027  1.2358 13.3191
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 29.59209    2.60749  11.349  < 2e-16 ***
## manu        -0.17211    0.03842  -4.480 1.38e-05 ***
## foreign      0.43641    0.20351   2.144  0.03344 *
## feemp       -0.31457    0.04824  -6.521 7.87e-10 ***
## black        0.19796    0.04548   4.353 2.33e-05 ***
## hisp        -0.49005    0.17300  -2.833  0.00518 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.064 on 168 degrees of freedom
## Multiple R-squared:  0.4827, Adjusted R-squared:  0.4673
## F-statistic: 31.35 on 5 and 168 DF,  p-value: < 2.2e-16
```

From the model summary output above, the selected model includes `manu`, `foreign`, `feemp`, `black`, and `hisp`. The coefficient for `feemp` remains negative, indicating that higher county-level employment is associated with lower poverty, after accounting for the other selected socio-economic factors.

We can perform Moran's I test on the residuals for the reduced model selected by backward elimination.

```
moran.test(resid(m1_back), listw_povW)
```

```
##
##  Moran I test under randomisation
##
## data:  resid(m1_back)
## weights: listw_povW
##
## Moran I statistic standard deviate = 3.2079, p-value = 0.0006686
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##       0.142732545      -0.005780347       0.002143353
```

From the reduced model, the Moran's I test statistic is positive and the p-value is small. At the 5% level, we reject the null hypothesis of CSR in the model residuals. There is strong evidence for spatial dependence in the residuals.

We can also perform the likelihood ratio test comparing the full and reduced models using the `anova()` function. We apply the function to the reduced model first and then to the full model, specifying `test = "Chisq"`.

```
anova(m1_back, m1, test = "Chisq")
```

```
## Analysis of Variance Table
##
## Model 1: povty ~ manu + foreign + feemp + black + hisp
## Model 2: povty ~ ag + manu + retail + foreign + feemp + hsch + black +
##     hisp
##   Res.Df    RSS Df Sum of Sq Pr(>Chi)
## 1    168 1577.6
## 2    165 1547.3  3    30.233   0.3584
```

From the analysis of variance table, the first model is the reduced model and the second model is the full model. The table reports the residual degrees of freedom and the residual sum of squares for each model,

along with the difference in degrees of freedom and the likelihood ratio test statistic. A large p-value indicates that the full model does not provide a significantly better fit than the reduced model.

Forward Selection

We can also perform forward variable selection. We use the function `step()` and specify `direction = "forward"`. We also specify the scope of the explanatory variables to be considered in the model.

```
m1_min <- lm(povty ~ 1, data = ilwipov_shape)

m1_forward <- step(m1_min,
                   direction = "forward",
                   scope = (~ ag + manu + retail + foreign + feemp + hsch + black + hisp))
```

```
## Start:  AIC=500.27
## povty ~ 1
##
##           Df Sum of Sq    RSS    AIC
## + feemp    1    862.76 2186.6 444.40
## + manu     1    733.69 2315.7 454.38
## + black    1    483.13 2566.2 472.26
## + hsch     1    165.33 2884.0 492.57
## <none>                 3049.3 500.27
## + hisp     1     26.08 3023.3 500.78
## + retail   1      7.35 3042.0 501.85
## + ag       1      6.51 3042.8 501.90
## + foreign  1      6.15 3043.2 501.92
##
## Step:  AIC=444.4
## povty ~ feemp
##
##           Df Sum of Sq    RSS    AIC
## + manu     1    374.27 1812.3 413.73
## + black    1    254.85 1931.7 424.84
## + hsch     1    219.15 1967.4 428.02
## <none>                 2186.6 444.40
## + foreign  1     14.88 2171.7 445.21
## + ag       1     11.69 2174.9 445.47
## + hisp     1      2.94 2183.7 446.17
## + retail   1      1.06 2185.5 446.32
##
## Step:  AIC=413.73
## povty ~ feemp + manu
##
##           Df Sum of Sq    RSS    AIC
## + black    1   153.402 1658.9 400.35
## + hsch     1    53.950 1758.4 410.48
## <none>                 1812.3 413.73
## + retail   1    14.441 1797.9 414.34
## + foreign  1     4.837 1807.5 415.27
## + ag       1     2.395 1809.9 415.50
## + hisp     1     1.262 1811.0 415.61
##
## Step:  AIC=400.35
## povty ~ feemp + manu + black
```

```
##
##           Df Sum of Sq    RSS    AIC
## + hisp     1    38.169 1620.7 398.30
## <none>                  1658.9 400.35
## + ag       1     8.560 1650.3 401.45
## + foreign  1     6.001 1652.9 401.72
## + retail   1     5.850 1653.1 401.73
## + hsch     1     3.248 1655.7 402.00
##
## Step:  AIC=398.3
## povty ~ feemp + manu + black + hisp
##
##           Df Sum of Sq    RSS    AIC
## + foreign  1    43.179 1577.6 395.60
## + hsch     1    23.646 1597.1 397.74
## <none>                  1620.7 398.30
## + retail   1     7.914 1612.8 399.44
## + ag       1     0.861 1619.9 400.20
##
## Step:  AIC=395.6
## povty ~ feemp + manu + black + hisp + foreign
##
##          Df Sum of Sq    RSS    AIC
## <none>                1577.6 395.60
## + hsch    1    8.0216 1569.5 396.71
## + retail  1    5.5594 1572.0 396.98
## + ag      1    4.9099 1572.7 397.05
```

From the output above, we see that the stepwise regression selected the same variables as did backward elimination. The AIC in this final model is the same as that from backward elimination.